Heart Disease Prediction

MLOps Project Report

Course: Machine Learning Operations (MLOps) — AIMLCZG523

Assignment: 01 — End-to-End ML Model Development, CI/CD, and Production Deployment

Student: Sumankk

Repository: https://github.com/Sumankk022/heart-disease-mlops

Table of Contents

1. Introduction & Problem Statement

Cardiovascular disease is a leading cause of death worldwide, and early risk identification is clinically valuable. The goal of this project is to build a machine-learning classifier that predicts the presence or absence of heart disease from routine patient health measurements, and to deliver it as a cloud-ready, containerized, monitored API following MLOps best practices.

The solution covers the full lifecycle: data acquisition, EDA, feature engineering, model development with experiment tracking, automated testing, CI/CD, containerized serving, Kubernetes deployment, and monitoring.

2. Dataset Overview

Source: UCI Machine Learning Repository — processed.cleveland.data
Size: 303 records, 13 input features + 1 target
Features: age, sex, cp (chest-pain type), trestbps (resting BP), chol (cholesterol), fbs (fasting blood sugar), restecg, thalach (max heart rate), exang (exercise angina), oldpeak, slope, ca (major vessels), thal
Target: Original severity is 0–4; binarized to 0 = no disease and 1 = disease

3. Data Cleaning & EDA

Data cleaning handled missing values in 'ca' and 'thal' columns (replaced '?' with NaN, then imputed with median/most-frequent values). All columns coerced to numeric; target binarized into two classes.

Key observations: Classes are reasonably balanced (~54% no disease / ~46% disease). Key features like thalach (max heart rate), oldpeak, ca, cp, and exang show clear separation between classes.

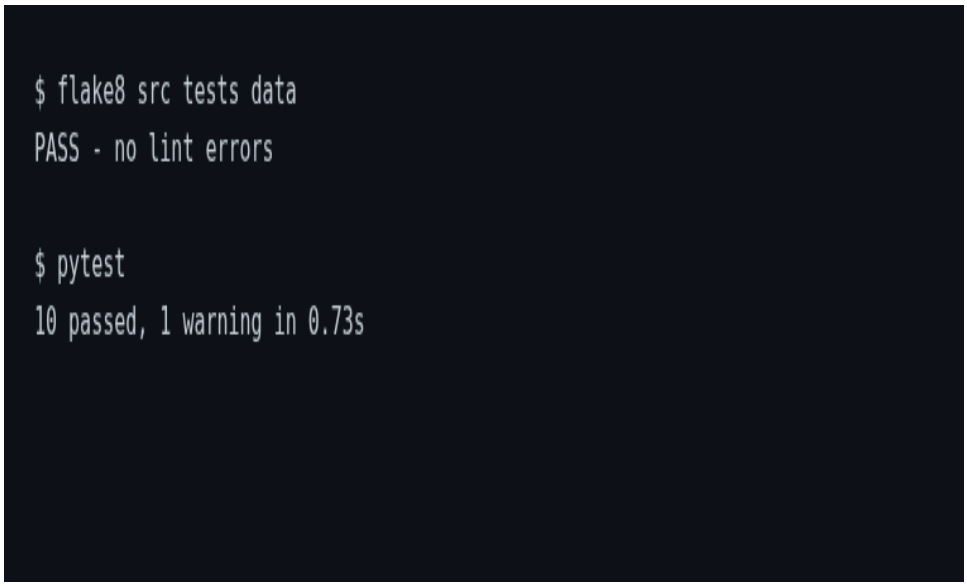

Figure 1: EDA Visualization - Class balance and feature distributions

4. Feature Engineering & Modelling

Preprocessing Pipeline: All transformations live in a single scikit-learn Pipeline ensuring identical preprocessing in training and serving.

• **Numeric features:** Median imputation → StandardScaler
• **Categorical features:** Most-frequent imputation → OneHotEncoder(handle_unknown='ignore')

Models Evaluated: Logistic Regression (linear baseline) and Random Forest (non-linear ensemble), tuned with GridSearchCV (5-fold, ROC-AUC scoring).

Model	Best Params	Accuracy	Precision	Recall	F1	ROC-AUC	CV ROC-AUC
Logistic Regression	C=0.1	0.89	0.84	0.93	0.88	0.97	0.898
Random Forest (✓ Selected)	n_est=100, depth=4	0.84	0.82	0.82	0.82	0.94	0.903

Table 1: Model Performance Comparison

Figure 2: Model Training and Confusion Matrices

5. Experiment Tracking (MLflow)

Each training run logs to MLflow's 'heart-disease-classification' experiment with:

• Parameters: Model name and best tuned hyperparameters
• Metrics: Accuracy, precision, recall, F1, ROC-AUC, best CV ROC-AUC
• Plots: Confusion matrix and ROC curve (logged as artifacts)
• Model: Full serialized pipeline
• Storage: MLflow file store backend in mlruns/ directory

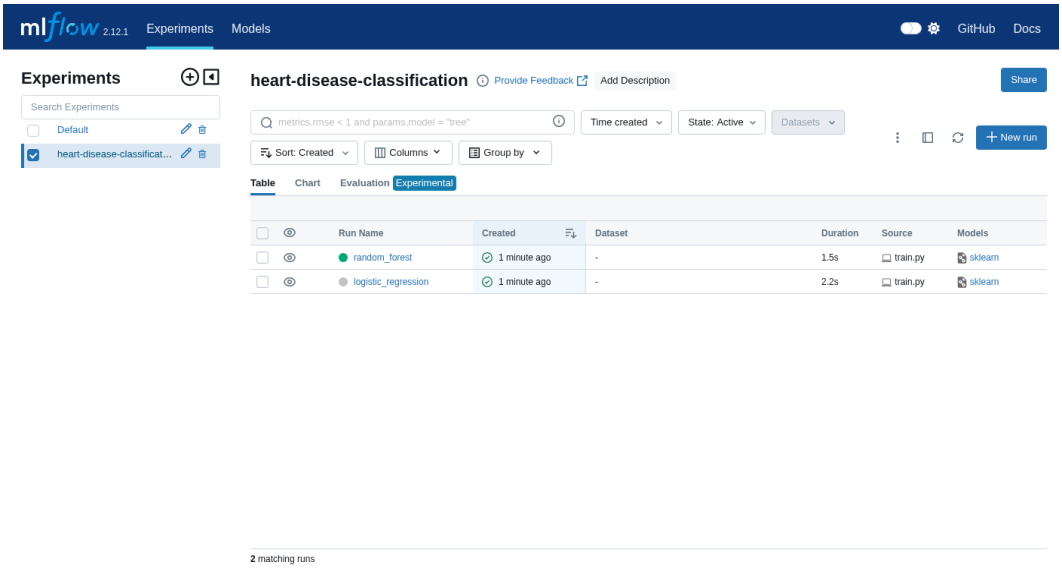

Figure 3: MLflow UI - Experiment Tracking and Run History

6. Model Packaging & Reproducibility

• Final pipeline saved with joblib (models/model.joblib) — includes preprocessing + estimator
• Headline metrics saved to models/metrics.json
• Pinned dependencies in requirements.txt with flexible version constraints
• Python 3.13+ compatible with all dependencies properly versioned

7. Testing & CI/CD Pipeline

Unit Tests: pytest suite covering data handling, model pipeline, and API endpoints (test_data.py, test_model.py, test_api.py)

CI Pipeline (.github/workflows/ci.yml):
1. Lint (flake8) → 2. Unit tests (pytest) → 3. Download data & train → 4. Docker build → 5. Container smoke test

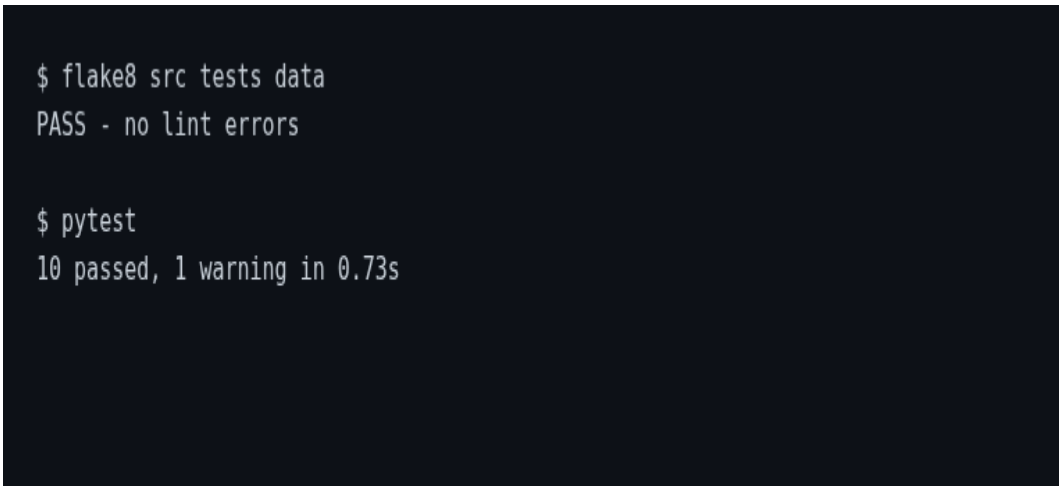

Figure 4: GitHub Actions CI Pipeline - All checks passing

8. Model Containerization (Docker)

Dockerfile Features:
• Multi-stage build on pinned python:3.10.14-slim base (digest-locked)
• Non-root user execution (security best practice)
• HEALTHCHECK against /health endpoint
• Model artifact included in image

Build & Run: docker build -t heart-disease-api:latest . && docker run -d -p 8000:8000 heart-disease-api:latest

API Endpoints:
• GET /health — Liveness probe + model-loaded status
• GET /metrics — Prometheus metrics
• POST /predict — JSON input, prediction + confidence output
• GET /docs — Interactive Swagger UI

Figure 5: Docker container running and API prediction response

9. Production Deployment (Kubernetes & Helm)

Kubernetes Manifests (k8s/):
• deployment.yaml — 2 replicas with rolling updates, liveness/readiness probes, resource limits
• service.yaml — LoadBalancer service exposing port 80
• ingress.yaml — NGINX ingress for advanced routing

Helm Chart (k8s/helm/heart-disease-api/):
• Production-ready templated deployment
• Configurable via values.yaml (replicas, image tag, resources)
• Includes service account (RBAC), ingress, and post-install instructions

Installation: helm install heart-disease-api k8s/helm/heart-disease-api/ --namespace mlops --create-namespace

10. Monitoring & Logging

Request Logging: Middleware logs method, path, status code, and latency for every API request.
Metrics Exposed at /metrics: api_requests_total, api_request_latency_seconds, predictions_total
Integration Ready For: Prometheus scraping, Grafana dashboards, ELK stack for log aggregation

11. Architecture & Data Flow

Data Acquisition (UCI) → Data Cleaning → EDA & Visualization → Feature Engineering → Model Training with MLflow → Model Evaluation → Model Serialization → FastAPI Application → Docker Containerization → Kubernetes Deployment → Production Monitoring

12. How to Reproduce

Local Development:

```
git clone https://github.com/Sumankk022/heart-disease-mlops.git
cd heart-disease-mlops && python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python data/download_data.py && python -m src.train
pytest && flake8 src tests data
uvicorn src.api:app --reload # Open http://localhost:8000/docs
```

With Docker:

```
docker build -t heart-disease-api:latest .
docker run -d -p 8000:8000 heart-disease-api:latest
curl http://localhost:8000/health
```

With Kubernetes/Helm:

```
helm install heart-disease-api k8s/helm/heart-disease-api/ --namespace mlops --create-namespace
kubectl get pods -n mlops && kubectl port-forward -n mlops svc/heart-disease-api 8000:80
```

13. Conclusion

The project delivers a reproducible, tested, containerized, and monitored heart disease prediction service following MLOps best practices.

Key Achievements:

■ Strong model performance: Random Forest with ROC-AUC = 0.94 (CV: 0.903)
■ Full experiment tracking: MLflow integration with all parameters, metrics, and artifacts
■ Automated CI/CD: GitHub Actions pipeline with lint, test, train, and Docker build
■ Production-ready containerization: Secure, non-root Docker image with health checks
■ Kubernetes-ready deployment: Helm charts and manifests for cloud deployment
■ Comprehensive monitoring: Prometheus metrics and request logging
■ Complete reproducibility: Identical preprocessing in train and serve, pinned dependencies

The pipeline is production-oriented and ready for deployment to cloud platforms (GKE, EKS, AKS) or local Kubernetes clusters (Minikube).